

英语背诵与学习软件 · 设计文档

版本 v1.0 · 2026-09-26 · 对应代码提交 0f2a009

相关文档: [需求文档-2026-09-03](#) · [说明书-2026-09-05](#) (使用手册) · [README](#) (交付形态与快速开始)

本文描述**如何实现**; 需求与功能边界见需求文档与 README。

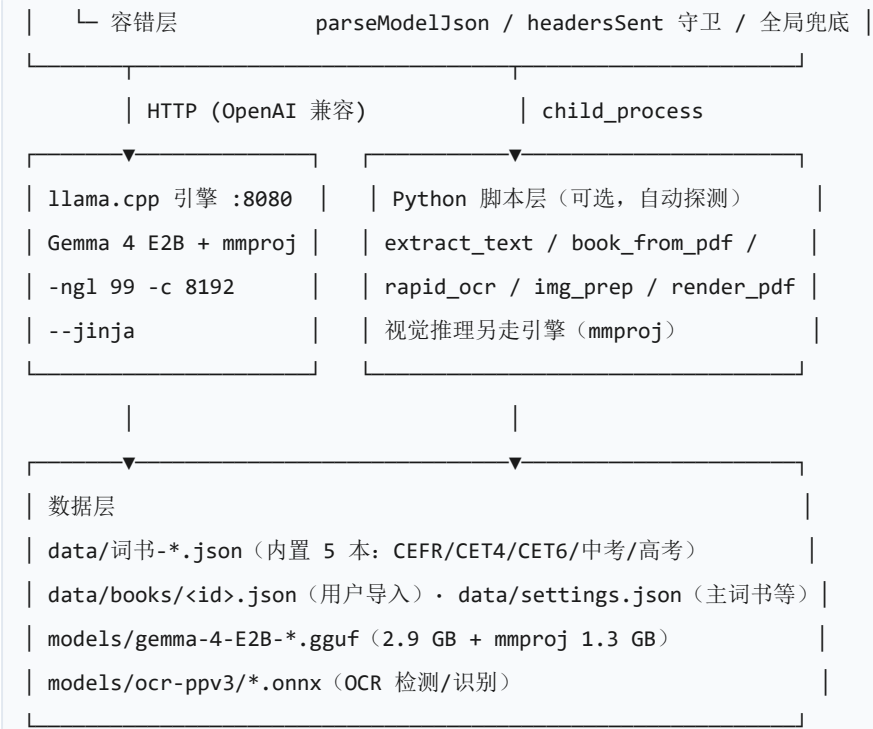
1. 设计目标与约束

约束	来源	设计后果
数据不出机器	学校对题目/试卷/作文的隐私诉求	模型本机推理, 服务只监听 127.0.0.1, 无任何上传路径
断网可用	校园网受限、机房不联网	模型、引擎、运行时全部随包分发, 无账号、无 API Key
消费级硬件可跑	学生笔记本 (实测 RTX 5060 Laptop 8GB)	Gemma 4 E2B (Q4_K_M, 约 2.9 GB) + llama.cpp; 启动前显存自检, 不足则自动降级 GPU 层数
零依赖后端	让"陌生人 5 分钟能跑"	Node 单文件服务, 不使用任何 npm 依赖 (仅 Node 内置模块)
非技术用户能用	学生与老师	以封装桌面软件交付 (双击即用), 源码另供开发者
可解释、可复现	学习结论要能被复核	归属判定交给规则层而非模型; 词书由脚本从明确许可的数据源重建

非目标: 多用户/账号体系、云端同步、听力训练 (见 §12)。

2. 总体架构





2.1 运行时拓扑 (三个进程)

进程	端口	职责	崩溃影响
llama-server.exe	8080	文本提炼、视觉识别 (mmproj) 、判卷补义	提炼不可用；服务端回退 Ollama (若在跑) ， 否则报错并提示
node server.js	8804	全部业务逻辑与静态资源	网页不可用 (已加全局未捕获兜底, 见 §9)
浏览器 / WebView2 壳	—	界面与本地状态	—

两个端口都只监听 127.0.0.1 ， 不对局域网暴露。

3. 目录结构与职责

app/	
server.js	零依赖 HTTP 服务 (路由/提炼/分层/词书/图像链/容错)
llama/	llama.cpp 引擎与运行库
public/	前端 (index.html / app.js / style.css)
scripts/	生产脚本 (可达执行)
parse_*.py	词书构建: 从公开数据源生成 data/词书-*.json
extract_text.py	PDF/DOCX/TXT → 文本 (扫描版返回 type=scan)
book_from_pdf.py	PDF 词汇手册 → 词表 (表格优先, 退化逐行)
rapid_ocr*.py	图片 OCR 快路径
img_prep.py / crop_*.py	图片预处理与裁剪
render_pdf.py	扫描版 PDF 逐页渲染 (供视觉识别)
verify_e2e.js	端到端回归 (20 项)

verify_books.js	词书系统验证（18 项）
_archive/	一次性脚本归档（不参与运行）
data/	词书与用户数据
models/	模型权重与 OCR onnx
docs/	文档（作品介绍、本设计文档、项目卡）

4. 核心模块设计

4.1 输入链路（三条，互不阻塞）

输入	路径	依赖	降级
文本粘贴	前端 → /api/extract → 引擎	仅引擎	引擎不可用 → 提示启动引擎
图片	前端 → /api/file-text → 图像处理后交引擎 (mmproj 视觉)	引擎（视觉）	Python 缺席不影响视觉路径
PDF / Word	前端 → /api/file-text → extract_text.py (pymupdf / docx)	Python + pymupdf	无 Python → 明确提示"可用文本粘贴或拍照"

图片处理链（§6 函数清单 中 12 个函数）：先在服务端做预处理与裁剪（`prepImage / cropTop / cropBand`），识别后用**乱码检测**（`isGarbled / badishLine / countLeadBad / countTailBad`）判定质量，再走**头尾修补**（`refineHeadTail / topStripRecovery / v4Supplement / visionStripSupplement`），直到文本可用。这是为了对付真实拍照的常见问题：倾斜、裁到一半、页眉页脚混入、低对比。

4.2 提炼模块

- **提示词**：`buildSystemPrompt(grade)` 按年级注入要求；输出固定 JSON 五板块 `vocabulary / phrases / golden_sentences / collocations / grammar`。
- **长文分块**：`splitText`，单块阈值 `MAX = 2000` 字符；`extractData` 分块提炼后**按 key 去重合并**（词形 / 词组 / 句子 / 搭配 / 语法点各一套 Set）。
- **健壮解析**（关键容错）：`parseModelJson` 逐级修复——
 1. 去 markdown 围栏 → 直接 `JSON.parse`
 2. 取首个 { 起、末个 } 止
 3. 扫描字符串状态与括号栈，在**最后一个结构完整处**截断并补齐闭括号
 4. 仍失败则回退丢弃尾部残片重试
 5. 全部失败 → 提高 token 预算（4500 → 6000）**自动重试一次**
- **token 预算**：默认 2000（兼容旧调用），提取用 **4500**，重试用 **6000**；判卷补义 3500，视觉识别 3000–4000。

4.3 候选分层引擎（本项目的核心算法）

`buildCandidates(text, annotated, grade, validBooks)` 产出学生自评的候选池，**三层来源**：

主词书（primary）	— <code>scanBook(text, grade)</code>	→ scope「考纲」
副词书（leveled）	— <code>scanLeveled(text, rec, grade)</code>	→ scope「CEFR」等

副词书 (list) —— scanList(text, rec, grade) → scope 「CET4/6」等
表外词 —— 引擎首轮产出的生词 → scope 「拓展」

归属判定 (规则层, 不由模型决定): 使用 buildBookIndex(book) 建立的索引——

- info: 词形 → 词条 (含**主形与括号内变体**: abandon、a(an)、take(off) 三类都建索引)
- phraseKeys: 词组集合 (归一化大小写与括号)

判定结果: 考纲 (在主词书) / 拓展 (不在任何词书) / 副词书层 (在副词书)。

年级门槛 (KEEP_LINE + LVL_N):

小学 A1 · 初中 A1 · 高一 A2 · 高二 B1 · 高三 B1 · 大学 B2; CEFR 数值 A1=1 ... C2=6。

scanLeveled 只排除**确定入门**的词 (LVL_N[maxLevel] < nKeep), 并把已在主词书的词让给主词书层。

熟词僻义识别: 当某词在 CEFR 记录中跨多个等级 (minLevel ≠ maxLevel), 若**低义项已被年级门槛覆盖 (说明学过) 而高义项未学**, 则保留——即"认识这个词, 但没学过这个义项"。这是同类工具普遍漏掉的一类考点。

语境补义: fillContextMeanings 对无释义的候选, 把候选词与原文 (截断) 交给引擎补义与例句, 避免"清单里只有词没有义"。

4.4 词书系统

- **注册表**: BOOKS: Map<id, {id, name, kind, book, index, path, removable}>; loadAllBooks() 启动时加载内置词书并**发现** data/ 下新增的词书文件。
- **内置 5 本**: cefr (CEFR 分级 7035 词) / cet4 (5183) / cet6 (5974) / zk 中考 (1600) / gk 高考 (3837)。
- **主词书**: PRIMARY_ID, 持久化在 data/settings.json; 未设置时优先 gk。
- **导入**: importBook 支持 ① 标准 JSON 词书 ② 裸数组 ③ 纯文本词表 (每行一词, 制表符/多空格/冒号分隔释义); importBookFromPdf 支持 **PDF 词汇手册** (见 §4.5)。导入落 data/books/<id>.json, removable=true。
- **切换主词书即换考纲**: /api/books/primary 后, 下一次提炼的「考纲」层即来自新主词书 (实测: 导入自定义 8 词词书 → 设为主词书 → 提炼的考纲层来自该书且释义取自该书)。

4.5 PDF 词汇手册导入

```
POST /api/books/import {name, filename, pdfBase64}
→ 落临时文件 → scripts/book_from_pdf.py --out <json>
→ {type:"ok", words[], stats} → importBook()
```

book_from_pdf.py 策略: **pymupdf find_tables()** **表格优先** (词汇手册多为表格), 取"最像单词"的列作词、含中文的列作释义; 表格不足 50 条时退化**逐行解析** (词 + 音标 [...] //.../ + 释义, 过滤页眉页码与章节标记)。文字层不足的**扫描版**返回 {type:"scan"}, 服务端给出可操作提示 (改用拍照识别)。

陷阱记录: find_tables 会向 stdout 打印提示且可能是 UTF-16, 故结果走 --out 文件、Python 侧全程转走 stdout。

4.6 默写与判卷

- **双向默写**：中→英（看释义写词）、英→中（看词写义）；支持**原文例句挖空**（把该词在原句位置替换为空格，提供上下文线索）。
- **判卷**：`/api/judge-zh` 交引擎做**容错判卷**（拼写近似、大小写、词形变化、中文近义表述均视为正确），避免机械字符串比对挫伤学习。
- **干扰项池**：`makeDistractors()` 从**注册表内任意词书**随机抽释义，供四选一题型（不再依赖任何单一词书）。

4.7 记忆层（前端为主）

- **SM-2 间隔复习**：按掌握度维护 `ease` 与间隔，到期自动进入当日复习队列。
- **掌握三分**：`未学 / 见过(seen) / 已掌握(master)`；`seen && !master` 计入"学习中"。
- **错题池**：默写/判卷判错自动入池，优先复现。
- **每日自测**：上限 ≤ 30 题，避免过载；`/api/read` 与 `/api/util` 提供朗读与工具能力。
- **本地存储**：状态存 `localStorage`（`es_streak` 连续天数、`es_grade` 年级、`es_books` 勾选词书、`es_persona` 语伴人格、`es_mem_v2_*` 掌握记录等）——**学习数据不落服务端**，是"数据不出机器"的最后一环。

4.8 聊天（语伴/作文）

`/api/chat/open`（开题/初始化）、`/api/chat`（对话，按 `es_persona` 切换语伴人格与纠错力度）、`/api/essay`（作文批改）。三者均走本机引擎。

5. 接口设计（18 个）

方法	路径	用途
POST	<code>/api/extract</code>	文本 → 五板块 + 候选池 + 干扰项
POST	<code>/api/file-text</code>	文件/图片 → 文本（PDF/DOCX/TXT/图片视觉）
POST	<code>/api/word-meanings</code>	批量取词义（候选补义）
POST	<code>/api/judge-zh</code>	中文答题容错判卷
POST	<code>/api/mode</code>	读写运行模式（ <code>data/settings.json</code> ）
GET	<code>/api/health</code>	引擎状态 + 词书摘要 + 主词书
GET	<code>/api/books</code>	词书列表（含 <code>primary / removable</code> 标记）
POST	<code>/api/books/import</code>	添加词书（JSON / 文本 / PDF）
POST	<code>/api/books/primary</code>	设为主词书
POST	<code>/api/books/delete</code>	删除用户词书
GET	<code>/api/book/<id></code>	翻看词书（ <code>letter= / q=</code> 搜索）
GET	<code>/api/word</code>	单词详情
POST	<code>/api/chat/open · /api/chat · /api/essay</code>	语伴对话 / 作文批改
POST	<code>/api/read · /api/util</code>	朗读 / 工具
GET	<code>/</code>	前端页面

统一约定：`Content-Type: application/json; charset=utf-8`；错误返回 `{ok:false, error}`；**请求体上限**：旧路由 8 MB，词书导入（含 PDF base64）60 MB。

6. 可靠性与容错设计（★ 实际踩过的坑都在这里）

风险	设计对策	触发动机
模型输出被截断/夹带说明	<code>parseModelJson</code> 逐级修复 + 提高预算自动重试 (\$4.2)	实测：一篇课文曾因 <code>max_tokens=2000</code> 截断， <code>JSON.parse</code> 抛错导致单次失败
响应头发送后再抛错	提取处理器改为 全部算完再 <code>writeHead</code> ； <code>catch</code> 增加 <code>res.headersSent</code> 守卫	实测：先发 200 再计算，中途抛错时 <code>catch</code> 二次 <code>writeHead</code> → <code>ERR_HTTP_HEADERS_SENT</code> 未捕获，整个服务进程退出
任何未捕获异常都不该让本地服务整体死亡	注册 <code>process.on('uncaughtException')</code> 与 <code>'unhandledRejection'</code> ，记录并保持存活	桌面软件没有“运维重启”，单请求失败不应演化成服务不可用
引擎未启动 / 崩溃	<code>engineAlive()</code> 探活；失败回退 <code>ollamaChat</code> （本机 Ollama）；均不可用则给出明确指引	—
显存不足	启动前 <code>nvidia-smi</code> 自检：空闲 <4.5 GB 时 <code>-ngl 25</code> 降级并把提示写进启动日志	避免与 ComfyUI 等 GPU 程序争抢导致崩溃
本机无 Python / 缺包	<code>findPy()</code> 探测链（内置 → 环境变量 → 常见 <code>venv</code> ）； <code>PY_RESOLVED</code> 为空时文档解析与 OCR 降级， 文本与视觉路径 不受影响	让“没装环境的电脑”也能用核心功能
扫描版 PDF	<code>extract_text.py</code> 判定文字层过少 → <code>{type:"scan"}</code> ；服务端转译为可读提示	不静默失败

7. 性能与资源实测

项目	实测值	环境
引擎冷启动	8 秒	RTX 5060 Laptop 8GB
文本提炼（约 90 词短文）	13–15 秒	同上， <code>-ngl 99 -c 8192</code>
长文提炼（重复 9 段）	13–14 秒（含分块合并）	—
图片识别（6 KB 测试图）	13.9 秒	视觉路径（ <code>mmproj</code> ）
PDF 词汇手册解析（87 页）	4.6 秒 → 3311 词	表格模式
模型占用	2.9 GB（Q4_K_M）+ <code>mmproj</code>	显存
封装包体积	约 6.69 GB（含运行时与模型）	单文件夹免安装
源码仓库	34.9 MB / 94 文件	适合公开托管

回归测试：`scripts/verify_e2e.js`（端到端 20 项）· `scripts/verify_books.js`（词书 18 项）。

8. 安全与合规设计

- 1. **无上传路径**：服务只监听回环地址；代码中不存在向外部域名发送数据的调用；学习数据存 `localStorage` 。
- 2. **许可合规**：代码 Apache-2.0；第三方组件逐条实测上游许可（Gemma 4 = Apache-2.0 / llama.cpp = MIT / PaddleOCR·RapidOCR = Apache-2.0 / 词书数据源 = MIT），逐条列于 `THIRD_PARTY_NOTICES.md` 。
- 3. **版权隔离**：模型权重与引擎因体积不随仓库分发；**各地官方考纲词汇手册因版权不随仓库分发**——改为"由用户导入自有词书"（`/api/books/import` 支持 PDF/JSON/文本），这是产品能力而非妥协。
- 4. **不采集**：无账号、无遥测、无广告。

9. 已知限制（如实列出）

限制	现状	计划
双栏排版图片的列切分	未做（单栏优先）	依据版面检测切列
长词跨行截断（PDF 表格）	少数长词（如 <code>comprehension</code> ）解析不到	单元格碎片拼接（需防误粘）
学习统计报表	约 30%	周/月掌握曲线与薄弱项聚类
数据导出、老师端、移动端	未实现	见 README 后续规划
听力训练	未覆盖 （能力边界：覆盖除听力外各环节）	未列入本期

10. 附录

10.1 关键常量

```
PORT = 8804                ENGINE_PORT = 8080
MAX = 2000                 （单块字符阈值）
KEEP_LINE = {小学:A1, 初中:A1, 高一:A2, 高二:B1, 高三:B1, 大学:B2}
LVL_N      = {A1:1, A2:2, B1:3, B2:4, C1:5, C2:6}
BUILTIN_BOOKS = cefr / cet4 / cet6      （+ zk/gk 启动时自动发现）
```

10.2 server.js 函数清单（56）

索引与词书	<code>buildBookIndex</code> <code>loadCet</code> <code>readSettings</code> <code>writeSettings</code> <code>slugId</code> <code>inferKind</code> <code>registerBook</code> <code>loadAllBooks</code> <code>primaryIndex</code> <code>bookBrief</code> <code>parseTextWordList</code> <code>importBookFromPdf</code> <code>importBook</code>
分层引擎	<code>scanList</code> <code>scanLeveled</code> <code>scanBook</code> <code>scanCefr</code> <code>scanCet</code> <code>passKeep</code> <code>buildCandidates</code> <code>annotate</code>
提炼	<code>buildSystemPrompt</code> <code>llamaChat</code> <code>ollamaChat</code> <code>extractFromText</code> <code>splitText</code> <code>parseModelJson</code> <code>extractData</code> <code>fillContextMeanings</code> <code>makeDistractors</code>
输入与图像	<code>isGarbled</code> <code>ocrImage</code> <code>findPy</code> <code>saveMode</code> <code>ocrKickIdle</code> <code>ensureOcr</code> <code>prepImage</code> <code>engineAlive</code> <code>dedupeLines</code> <code>removeJunkLines</code> <code>visionReadFiles</code> <code>cleanupVisionText</code> <code>ocrFast</code> <code>visionTopPassage</code> <code>cropTop</code> <code>topStripRecovery</code> <code>cropBand</code> <code>badishLine</code> <code>countLeadBad</code> <code>countTailBad</code> <code>headTailBad</code> <code>v4Supplement</code>

	visionStripSupplement refineHeadTail
判卷/工具	splitSent findSent

本设计文档随代码演进维护；实现与本文不一致时以代码为准。